

ROS 2

Advanced communication I

Roberto Masocco

`roberto.masocco@uniroma2.it`

Tor Vergata University of Rome

Department of Civil Engineering and Computer Science Engineering

May 7, 2026



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Recap

ROS 2 software is organized in **packages**, built by `colcon` invoking either **CMake** or **setuptools**.

Messages are the most basic, **one-way** communication paradigm.

In the **DDS RMW**, ROS 2 topics resolve to **DDS topics**.

In the **Zenoh RMW**, ROS 2 topics resolve to **Zenoh keys**.

Messages formats are defined in **interface files** which usually constitute entire packages.

This lecture is [here](#).

Roadmap

1 Interface packages

2 Asynchronous I/O

3 Services

Roadmap

1 Interface packages

2 Asynchronous I/O

3 Services

Defining custom interfaces

From package organization to build

Custom communication interfaces can be defined in **appropriate packages**.

The organization of such packages is regulated by a set of **best practices**.

When these packages are built, they generate both **code to include in**, and **libraries to link against**, the packages using them.

For everything to work, each package must have **the same interface packages** as **dependencies**, as well as link against **the same generated code**.

Find all relevant information in our [interfaces.md](#) file!

Roadmap

1 Interface packages

2 Asynchronous I/O

3 Services

What is I/O?

Informal scheduling-level definition

In an **operating system kernel**, a **task** (specifically, a **thread**) in userspace may perform operations pertaining to these two broad families:

- execute **computations** (e.g., $1 + 1 = 2$), using regular **CPU** instructions;
- access **system resources** (both **hardware** and **software**) through **system calls**, **exchanging data** in both directions.

When the kernel offers an interface¹ to said resources, **I/O** (*Input/Output*) denotes the act of using them performed by a thread.

OS schedulers typically distinguish between **CPU-bound** and **I/O-bound** tasks, because of their different **execution patterns**.

¹Drivers, protocols, software stacks...

Blocking I/O

What the kernel likes the most

The most common execution pattern for a task that performs an I/O system call goes like this:

- ① prepare the **input data** for the system call;
- ② call an **API** that performs the system call;
- ③ the kernel **blocks the task**, which stays **waiting** for the operations to complete;
- ④ the kernel **returns control** to the task when the system call is completed;
- ⑤ **output data**, returned by the kernel, can be accessed by the task.

This is **blocking I/O**, because the task is **blocked** while waiting for the system call to complete.

Examples of **blocking calls**: read, write to **file descriptors**.

Non-blocking I/O

What userspace applications like the most

If the kernel supports this feature, a task can perform a **non-blocking system call**:

- ① prepare the **input data** for the system call;
- ② call an **API** that performs the system call;
- ③ the kernel **returns control** to the task **immediately**, without blocking it;
- ④ the task can **poll** the system call **status** to check if it is completed;
- ⑤ when the system call is completed, the task can **access the output data**;
- ⑥ optionally, a userspace **callback** routine may be registered to be executed right when the system call is completed, with access to output data.

This is **non-blocking I/O** (or *asynchronous I/O*, or *overlapped I/O*), because the task is **not blocked** while waiting for the system call to complete, and things can happen in between.

Examples of **non-blocking calls**: read, write to **sockets** configured appropriately.

Non-blocking I/O

What userspace applications like the most

Usually, the **operation status** may be **inspected** through some kind of **handle object** returned by the API.

Some **programming languages** implement **future objects**: datatypes that hold the result of an asynchronous operation, which can be inspected to check if the operation is completed, and to retrieve the result once it is; **they are said to hold a value only when the operation is completed**.

ROS 2 makes a heavy use of **callbacks** and **future objects** to handle **asynchronous I/O**.

Roadmap

1 Interface packages

2 Asynchronous I/O

3 Services

ROS 2 services

Basic client-server paradigm

ROS 2 builds on the basic one-way messages adding two more communication paradigms: the first is the **service**. It allows nodes to establish quick and simple **client-server** communications.

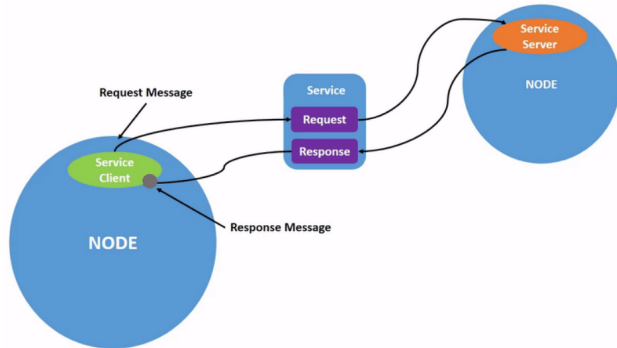


Figure 1: Two nodes acting as service *client* and *server*.

ROS 2 services

Communication overview

- ① The **client** sends a **request message** to the server.
- ② The **server** receives the request and processes it.
- ③ Meanwhile, the **client** may either **block** waiting for the response or **synchronously poll** it.
- ④ When done, the **server** sends a **response message** to the client.
- ⑤ If waiting, the **client** awakes when it receives the response.

The main command is `ros2 service` with the following verbs.

- `list` Lists all active services.
- `type` Prints the service type.
- `find` Lists active services of the given type.
- `call` Calls the service with the request defined in the command line.

ROS 2 services

Coding hints for servers and clients

Servers

Similarly to topic subscriptions, requests are processed in appropriate **callbacks**, taking **two arguments**: the **request** containing input data and the **response** to be filled with output data. The Service object is only needed to handle the life cycle of the service.

Clients

As per the previous dynamics, one has to **code each step** of the client side into their application using appropriate **ROS 2 APIs**. **The client object is used to send requests**, while **responses are handled as future objects^a**.

^a[std::future - C++ Reference](#)

Interface files

Services

The entire system is built on messages, so **combine two of them** in a single interface file, separated by ---.

Service file names end with `.srv`.

```
1 # REQUEST
2 int64 a
3 int64 b
4 ---
5 # RESPONSE
6 int64 sum
7
```

Listing 1: Definition of the `example_interfaces/srv/AddTwoInts` service.

Example

Simple service

Now go have a look at the [ros2-examples/src/cpp/simple_service_cpp](https://github.com/ros2/examples/tree/main/src/cpp/simple_service_cpp) package!

- Run the service client and server examples, and try to call the service from the command line.
- Add a timer to the subscriber in the `topic_pubsub_cpp` package to periodically toggle the ROS 2 subscriber on and off; how would you do that?
(Solution: [resetting_sub](#) example.)
- Create two new packages: one for server and client nodes, and one for a custom service definition named `CapString.srv`; the server should take a string as input and return two strings: the same string fully capitalized, and the number of characters in the string.
(Before trying this: read [interfaces.md](#).)